

# CSA1717-ARTIFICIAL INTELLIGENCE

## PRACTICAL LAB EXPERIMENTS

### RAGAVI S – (192565027)

#### EXPERIMENT 3: WATER – JUG PROBLEM

##### **Aim**

The aim of this experiment is to solve the Water Jug Problem using Breadth-First Search (BFS) to measure an exact target quantity of water using two jugs of different capacities.

##### **Objective**

To implement a state-space search algorithm to determine the shortest sequence of pouring, filling, and emptying operations required to reach a target water volume.

##### **Algorithm**

**Step 1:** Define the initial state as  $(0, 0)$  representing both jugs are empty, and the target goal state where either jug contains the target amount of water  $d$ .

**Step 2:** Initialize a queue to store the path of water states and a set to keep track of already visited states to prevent infinite loops.

**Step 3:** Push the starting state  $(0, 0)$  into the queue and mark it as visited.

**Step 4:** If the queue is empty, print that no solution exists and stop the program.

**Step 5:** Dequeue the current state  $(x, y)$  from the front of the queue.

**Step 6:** Check if the current state satisfies the goal condition (either  $x = d$  or  $y = d$ ). If yes, print the step-by-step path to this state and stop.

**Step 7:** Generate all possible next states by applying the valid water-pouring rules:

- Fill Jug 1 completely:  $(\text{Max1}, y)$
- Fill Jug 2 completely:  $(x, \text{Max2})$
- Empty Jug 1 completely:  $(0, y)$
- Empty Jug 2 completely:  $(x, 0)$
- Pour from Jug 1 to Jug 2 until Jug 2 is full or Jug 1 is empty.
- Pour from Jug 2 to Jug 1 until Jug 1 is full or Jug 2 is empty.

**Step 8:** For each newly generated state, check if it has been visited. If it is unvisited, mark it as visited, add it to the queue with its updated path, and return to **Step 5**.

## Source Code

```
from collections import deque
```

```
def solve(ca, cb, target, start_a):
```

```
    # Queue stores: (current_state, path)
```

```
    q = deque([((0, 0), [(0, 0)])])
```

```
    vis = {(0, 0)}
```

```
    while q:
```

```
        (a, b), path = q.popleft()
```

```
        if (a, b) == (target, 0):
```

```
            return path
```

```
    # Determine the next steps
```

```
    moves = [(ca, b), (a, cb), (0, b), (a, 0),
```

```
             (max(0, a + b - cb), min(cb, a + b)),
```

```
             (min(ca, a + b), max(0, a + b - ca))]
```

```
    # If it's the very first action, force our starting strategy choice
```

```
    if len(path) == 1:
```

```
        moves = [(ca, 0)] if start_a else [(0, cb)]
```

```
    for nxt in moves:
```

```
        if nxt not in vis:
```

```
            vis.add(nxt)
```

```
            q.append((nxt, path + [nxt]))
```

```
# --- Run & Print Both Paths ---

for start_a in [False, True]:

    label = "JUG B (3L)" if not start_a else "JUG A (4L)"

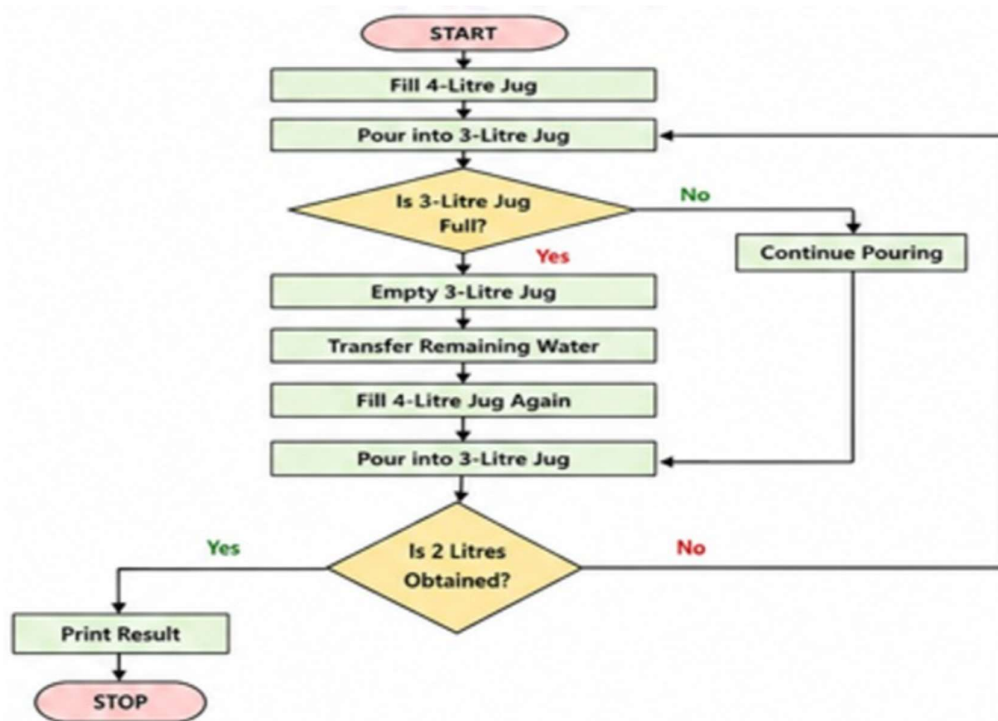
    path = solve(4, 3, 2, start_a)

    print(f"=== STARTING WITH {label} ===")

    print(" -> ".join(map(str, path)))

    print(f"Total steps: {len(path) - 1}\n")
```

## Flowchart



## Output

```
>>>
==== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/EXPl.py ====
=== STARTING WITH JUG B (3L) ===
(0, 0) -> (0, 3) -> (3, 0) -> (3, 3) -> (4, 2) -> (0, 2) -> (2, 0)
Total steps: 6

=== STARTING WITH JUG A (4L) ===
(0, 0) -> (4, 0) -> (1, 3) -> (1, 0) -> (0, 1) -> (4, 1) -> (2, 3) -> (2, 0)
Total steps: 7

>>>|
```

## **Result**

The Water Jug problem was successfully solved using the Breadth-First Search (BFS) algorithm by finding the optimal sequence of steps to measure the exact target volume of water.